

Relatório Tarefa 3: Implementação, Análise de Convergência e Custo Computacional no Cálculo de Pi

Thiago Jordão

30 de maio de 2026

1 Introdução

O objetivo desta experiência foi implementar e analisar algoritmos em linguagem C capazes de calcular aproximações da constante matemática π . O foco do estudo é avaliar o compromisso (*trade-off*) entre o tempo de processamento, o número de iterações e a precisão do resultado final, comparando duas abordagens matemáticas distintas.

Foram implementadas duas soluções:

1. **Série de Gregory-Leibniz:** Uma série infinita clássica e de implementação direta:

$$\pi = 4 \sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1}$$

2. **Fórmula de Machin (1706):** Uma abordagem otimizada que utiliza a expansão da Série de Taylor para a função arco-tangente:

$$\frac{\pi}{4} = 4 \arctan\left(\frac{1}{5}\right) - \arctan\left(\frac{1}{239}\right)$$

2 Metodologia

Os programas foram desenvolvidos na linguagem C, aferindo o tempo de CPU por meio da biblioteca `<time.h>` (`clock_t`) e calculando o erro absoluto com base na constante `M_PI` da biblioteca `<math.h>`. Os códigos-fonte completos encontram-se na seção de Anexos no final deste documento.

Para a Série de Leibniz, os testes variaram de 10 a 1.000.000.000 de iterações. Para a Fórmula de Machin, dada a sua característica matemática, os testes exploraram um espectro menor, de 1 a 10.000.000 de iterações. Todos os cálculos utilizaram o tipo `double` para garantir dupla precisão em vírgula flutuante.

3 Resultados e Análise Comparativa

Tabela 1: Convergência pela Série de Leibniz

Iterações	Tempo (s)	Valor Estimado	Erro Absoluto
10	0.000005	3.0418396189	0.0997530347
100	0.000001	3.1315929036	0.0099997500
10.000	0.000019	3.1414926536	0.0001000000
1.000.000	0.001634	3.1415916536	0.0000010000
100.000.000	0.188943	3.1415926436	0.0000000100
1.000.000.000	1.845652	3.1415926526	0.0000000010

Tabela 2: Convergência pela Fórmula de Machin

Iterações	Tempo (s)	Valor Estimado	Erro Absoluto
1	0.000002	3.1832635983	0.0416709447
2	0.000001	3.1405970293	0.0009956243
5	0.000001	3.1415926824	0.0000000288
10	0.000001	3.1415926536	0.0000000000
10.000.000	0.059722	3.1415926536	0.0000000000

Análise do Custo vs. Precisão: Os dados empíricos demonstram claramente a diferença entre depender do poder de processamento bruto e utilizar a otimização matemática adequada:

- **Comportamento Linear vs. Exponencial:** A série de Leibniz (Tabela 1) apresenta uma convergência extremamente lenta. Para reduzir o erro absoluto num fator de 10 (ganhar uma casa decimal de precisão), é estritamente necessário multiplicar o número de iterações e o tempo de CPU por 10. Para atingir 9 casas decimais, o algoritmo levou ~ 1.84 segundos executando 1 milhão de milhões (bilião) de ciclos.
- **A Superioridade Algorítmica:** Em total contraste, a Fórmula de Machin (Tabela 2) aniquila a necessidade de força bruta. Com apenas **10 iterações** (executadas em ~ 0.000001 s), o erro absoluto cai a zero em relação à constante de referência, atingindo o limite físico da precisão do tipo `double` na linguagem C.

4 Reflexão sobre Aplicações Reais

A experiência comprova que alcançar maior acurácia exige invariavelmente mais esforço computacional. Contudo, ilustra também a “Lei dos Rendimentos Decrescentes”. Este comportamento iterativo repete-se sistematicamente na engenharia de software do mundo real e na concepção de sistemas complexos:

- **Inteligência Artificial e Machine Learning:** O treino de redes neurais baseia-se na minimização de uma função de perda de forma iterativa (ex: *Gradient Descent*). Nos ciclos iniciais, o modelo aprende rapidamente. No entanto, os ganhos

tornam-se marginais posteriormente. Exigir 1% a mais de acurácia num modelo de classificação pode significar multiplicar exponencialmente o volume de dados e o tempo de GPU, resultando em altos custos de infraestrutura e energia sem garantias de retorno prático (*overfitting*). A técnica de *Early Stopping* é o equivalente a parar nas 10 iterações da Fórmula de Machin.

- **Simulações Físicas (Dinâmica de Fluidos, Clima, Engenharia):** Simulações dependem da integração numérica ao longo do tempo. Calcular com passos amplos (poucas iterações) é rápido, mas acumula erros sistêmicos. Reduzir o passo (milhões de iterações) eleva a precisão, mas exige a alocação de supercomputadores durante dias. O engenheiro ou arquiteto de software deve sempre calcular o limite aceitável de tolerância para a tomada de decisão no mundo físico.
- **Computação Gráfica e *Ray Tracing*:** No algoritmo de *Path Tracing* para realismo visual, a cor de cada pixel é determinada pelo lançamento de milhares de “raios de luz” simulados. Para remover o ruído visual de uma cena, o número de amostras (iteraões) tem de ser massivo. Um ganho subtil no fotorrealismo de um único *frame* pode elevar o tempo de renderização de minutos para horas.

5 Conclusão

A comparação direta entre as séries matemáticas comprova um princípio fundamental da computação: os ganhos iniciais de precisão costumam ser baratos, mas alcançar o limite extremo da exatidão consome exponencialmente mais tempo, recursos de hardware e energia. Mais do que simplesmente aplicar ciclos de processamento a um problema, a escolha arquitetural do algoritmo correto (como a Fórmula de Machin em detrimento de Leibniz) é o que define a verdadeira escalabilidade de um sistema. Cabe aos engenheiros avaliar e decidir rigorosamente qual o nível de precisão satisfatório face aos recursos disponíveis.

A Anexos: Códigos-Fonte

A.1 Algoritmo pi_leibniz.c (Série de Gregory-Leibniz)

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4  #include <time.h>
5
6  #ifndef M_PI
7  #define M_PI 3.14159265358979323846
8  #endif
9
10 double calculate_pi(long long iterations) {
11     double pi_approx = 0.0;
12     int sign = 1;
13     for (long long i = 0; i < iterations; i++) {
14         pi_approx += sign * (1.0 / (2.0 * i + 1.0));
15         sign = -sign;
16     }
17     return pi_approx * 4.0;
18 }
19
20 int main() {
21     long long iter_counts[] = {10, 100, 1000, 10000, 100000, 1000000, 10000000, 100000000, 1000000000};
22     int num_tests = sizeof(iter_counts) / sizeof(iter_counts[0]);
23
24     printf("%-15s | %-15s | %-15s | %-15s\n", "Iterações", "Tempo (s)", "Valor Estimado", "Erro Absoluto");
25     printf("-----\n");
26
27     for (int i = 0; i < num_tests; i++) {
28         long long iters = iter_counts[i];
29
30         clock_t start_time = clock();
31         double pi_approx = calculate_pi(iters);
32         clock_t end_time = clock();
33
34         double elapsed_time = (double)(end_time - start_time) / CLOCKS_PER_SEC;
35         double error = fabs(M_PI - pi_approx);
36
37         printf("%-15lld | %-15.6f | %-15.10f | %-15.10f\n", iters, elapsed_time, pi_approx, error);
38     }
39
40     printf("-----\n");
41     printf("Valor real de Pi: %.15f\n", M_PI);
42
43     return 0;
44 }
45
```

Figura 1: Código-fonte da implementação da Série de Leibniz

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4  #include <time.h>
5
6  #ifndef M_PI
7  #define M_PI 3.14159265358979323846
8  #endif
9
10 double calculate_pi(long long iterations) {
```

```

11     double pi_approx = 0.0;
12     int sign = 1;
13     for (long long i = 0; i < iterations; i++) {
14         pi_approx += sign * (1.0 / (2.0 * i + 1.0));
15         sign = -sign;
16     }
17     return pi_approx * 4.0;
18 }
19
20 int main() {
21     long long iter_counts[] = {10, 100, 1000, 10000, 100000, 1000000,
22     10000000, 100000000, 1000000000};
23     int num_tests = sizeof(iter_counts) / sizeof(iter_counts[0]);
24     printf("%-15s | %-15s | %-15s | %-15s\n", "Iteracoes", "Tempo (s)",
25     "Valor Estimado", "Erro Absoluto");
26     printf("
27     -----\n");
28     for (int i = 0; i < num_tests; i++) {
29         long long iters = iter_counts[i];
30         clock_t start_time = clock();
31         double pi_approx = calculate_pi(iters);
32         clock_t end_time = clock();
33
34         double elapsed_time = (double)(end_time - start_time) /
35         CLOCKS_PER_SEC;
36         double error = fabs(M_PI - pi_approx);
37
38         printf("%-15lld | %-15.6f | %-15.10f | %-15.10f\n", iters,
39         elapsed_time, pi_approx, error);
40     }
41     printf("
42     -----\n");
43     printf("Valor real de Pi: %.15f\n", M_PI);
44     return 0;
45 }

```

A.2 Algoritmo pi_machin.c (Fórmula de Machin)

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4  #include <time.h>
5
6  #ifndef M_PI
7  #define M_PI 3.14159265358979323846
8  #endif
9
10 // Função para calcular arctan(x) usando a série de Taylor
11 // arctan(x) = x - x^3/3 + x^5/5 - x^7/7 + ...
12 double taylor_arctan(double x, long long iterations) {
13     double result = 0.0;
14     double x_squared = x * x;
15     double power = x;
16     int sign = 1;
17
18     for (long long i = 0; i < iterations; i++) {
19         result += sign * (power / (2.0 * i + 1.0));
20         power *= x_squared; // Próxima potência de x para o termo seguinte
21         sign = -sign;
22     }
23
24     return result;
25 }
26
27 double calculate_pi_machin(long long iterations) {
28     // Fórmula de Machin: pi / 4 = 4 * arctan(1/5) - arctan(1/239)
29     // pi = 16 * arctan(1/5) - 4 * arctan(1/239)
30     double term1 = taylor_arctan(1.0 / 5.0, iterations);
31     double term2 = taylor_arctan(1.0 / 239.0, iterations);
32
33     return 16.0 * term1 - 4.0 * term2;
34 }
35
36 int main() {
37     // A série de Machin converge absurdamente mais rápido que Leibniz.
38     long long iter_counts[] = {1, 2, 3, 5, 10, 20, 100, 1000, 10000, 1000000};
39     int num_tests = sizeof(iter_counts) / sizeof(iter_counts[0]);
40
41     printf("Aproximação de Pi pela Fórmula de Machin\n");
42     printf("%-15s | %-15s | %-15s | %-15s\n", "Iterações", "Tempo (s)", "Valor Estimado", "Erro Absoluto");
43     printf("-----\n");
44
45     for (int i = 0; i < num_tests; i++) {
46         long long iters = iter_counts[i];
47
48         clock_t start_time = clock();
49         double pi_approx = calculate_pi_machin(iters);
50         clock_t end_time = clock();
51
52         double elapsed_time = (double)(end_time - start_time) / CLOCKS_PER_SEC;
53         double error = fabs(M_PI - pi_approx);
54
55         printf("%-15lld | %-15.6f | %-15.10f | %-15.10f\n", iters, elapsed_time, pi_approx, error);
56     }
57
58     printf("-----\n");
59     printf("Valor real de Pi: %.15f\n", M_PI);
60
61     return 0;
62 }
63
```

Figura 2: Código-fonte da implementação da Fórmula de Machin

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4 #include <time.h>
5
6 #ifndef M_PI
7 #define M_PI 3.14159265358979323846
8 #endif
9
10 // Funcao para calcular arctan(x) usando a serie de Taylor
11 // arctan(x) = x - x^3/3 + x^5/5 - x^7/7 + ...
12 double taylor_arctan(double x, long long iterations) {
13     double result = 0.0;
14     double x_squared = x * x;
15     double power = x;
16     int sign = 1;
17
18     for (long long i = 0; i < iterations; i++) {
19         result += sign * (power / (2.0 * i + 1.0));
20         power *= x_squared; // Proxima potencia de x para o termo
21         // seguinte
22         sign = -sign;
23     }
24
25     return result;
26 }
27
28 double calculate_pi_machin(long long iterations) {
29     // Formula de Machin: pi / 4 = 4 * arctan(1/5) - arctan(1/239)
30     // pi = 16 * arctan(1/5) - 4 * arctan(1/239)
31     double term1 = taylor_arctan(1.0 / 5.0, iterations);
32     double term2 = taylor_arctan(1.0 / 239.0, iterations);
33
34     return 16.0 * term1 - 4.0 * term2;
35 }
36
37 int main() {
38     // A serie de Machin converge absurdamente mais rapido que Leibniz.
39     long long iter_counts[] = {1, 2, 3, 5, 10, 20, 100, 1000, 100000,
40     100000000};
41     int num_tests = sizeof(iter_counts) / sizeof(iter_counts[0]);
42
43     printf("Aproximacao de Pi pela Formula de Machin\n");
44     printf("%-15s | %-15s | %-15s | %-15s\n", "Iteracoes", "Tempo (s)",
45     "Valor Estimado", "Erro Absoluto");
46     printf("
47     -----\n");
48
49     for (int i = 0; i < num_tests; i++) {
50         long long iters = iter_counts[i];
51
52         clock_t start_time = clock();
53         double pi_approx = calculate_pi_machin(iters);
54         clock_t end_time = clock();
55
56         double elapsed_time = (double)(end_time - start_time) /
57         CLOCKS_PER_SEC;

```

```

53     double error = fabs(M_PI - pi_approx);
54
55     printf("%-15lld | %-15.6f | %-15.10f | %-15.10f\n", iters,
56     elapsed_time, pi_approx, error);
57 }
58
59     printf("
60     -----\n");
61     printf("Valor real de Pi: %.15f\n", M_PI);
62
63     return 0;
64 }

```